

實作手冊 · 含提示語

居家長照跌倒偵測與自動追蹤系統

用 Claude Code 從零做出四鏡頭監控、AI 跌倒姿態偵測、雲台鎖定
追蹤、語音求救與 LINE 緊急通報。

EDITION

2026-10-07 · 第 1 版

AUDIENCE

資訊、電機、電子相關科系學生

PLATFORM

Windows 11 · Python · TP-Link Tapo C220

METHOD

與 AI 助理逐階段對話完成,不是抄程式碼

這份手冊怎麼用

- 照順序做,一次只做一個階段。每個階段都有「目標、提示語、驗收、地雷」四塊,做完驗收才進下一階段。
- 提示語可以直接複製貼上。灰底框裡的【虛線框】是要換成你自己資料的位置。提示語已把老師踩過的坑寫進去,AI 就不會重蹈覆轍。
- 手冊沒有附完整程式碼——程式由你和 AI 對話產生。這樣你才會懂、也才能改成自己的版本。

本手冊依 2026 年 8 月「Tapo 居家長照跌倒監控與鎖定追蹤系統」實際開發對話整理,內容為真實踩坑經驗;圖片為合成示意圖,未使用真實住戶影像。

目錄

01	專案總覽與系統架構	做出來會長什麼樣、為什麼這樣選技術
02	事前準備	硬體、網路、攝影機帳號、軟體環境
03	與 AI 對話的六個原則	怎麼開口,比會不會寫程式重要
04	階段 0-2:開場、單鏡頭畫面、四分割儀表板	Level 1 基礎
05	階段 3-5:AI 姿態偵測、雲台控制、警報流程	Level 2 進階
06	階段 6-8:語音求救、鎖定追蹤、備份與驗收	Level 3 挑戰
07	提示語句型庫	通用、可套用到任何專題
08	除錯速查表	症狀 → 原因 → 處理
09	關鍵參數表與已知限制	調整時的依據
10	安全、隱私與使用界線	一定要讀

Level 1 · 基礎

階段 0-2。目標:四台鏡頭在同一個視窗穩定直播。建議 1-2 次課。

Level 2 · 進階

階段 3-5。目標:AI 判斷站、坐、跌倒;雲台可控;跌倒時自動拍照並通知。建議 2 次課。

Level 3 · 挑戰

階段 6-8。目標:語音求救、跌倒後鎖定追蹤。建議 2 次課。

課次為建議值(每次約 2 小時),實際依學生程度與網路環境調整。

專案總覽與系統架構

1.1 完成後能做什麼

- 四鏡頭即時直播:2×2 四分割視窗,每台可單獨啟動、停止、放大。
- AI 姿態偵測:標示畫面中的人數,並判斷「正常站立、正常坐姿、疑似倒地、跌倒倒地」。
- 雲台遙控:每台鏡頭有上下左右、停止、歸位、巡航按鈕。
- 跌倒警報:確認跌倒後自動拍照存檔、LINE 通知家人、電腦播放安撫語音;「解除警報」按鈕確認安全後才恢復偵測。
- 語音求救:用攝影機自己的麥克風聽,聽到「救命、求救、好痛」等詞或巨大聲響就觸發同一套警報。
- 跌倒鎖定追蹤:跌倒後鏡頭自動把人拉回畫面中央,人站起來走動也持續跟著;按「解除追蹤」或舉起右手即停止。

1.2 系統架構

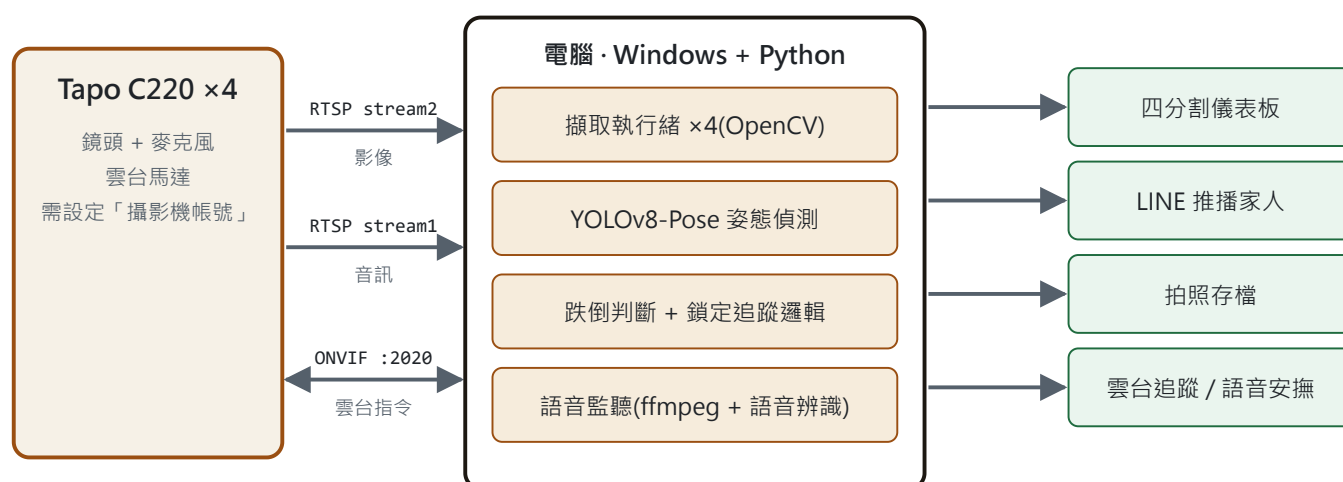


圖 1 資料流向。影像與音訊走 RTSP(兩條不同的串流),雲台控制走 ONVIF,三者都要連到「同一個區域網路」。

1.3 技術選擇與原因

需求	採用	為什麼
取得影像	RTSP + OpenCV(TCP 傳輸)	Tapo 內建、不用額外裝置;改走 TCP 才不會花屏、斷幀。
雲台控制	ONVIF(連接埠 2020, onvif-zeep)	2026 年 7 月 Tapo 韌體更新後,TP-Link 私有本機 API(pytapo)登入交握失敗;ONVIF 是通用標準,仍然可用。
姿態偵測	YOLOv8n-pose(ultralytics)	輕量、CPU 就能跑,直接輸出 17 個人體關鍵點(肩、髖、手腕...)。
跌倒判斷	軀幹角度 + 外框長寬比 + 持續 3 秒	只看一幀會把「彎腰撿東西」誤判成跌倒;要求持續躺平才算。
認得同一個人	衣服顏色的 HSV 直方圖比對	簡單、不用再訓練模型;人從躺到站、姿態改變仍然認得。
警報通知	LINE Messaging API(broadcast)	家人手機即時收到,免費方案有每月則數上限。
語音求救	ffmpeg 抓 RTSP 音訊 + SpeechRecognition	用「攝影機自己的麥克風」,聽得到各房間,而不是電腦旁邊的麥克風。



圖 2 AI 標籤的四種狀態(合成示意圖,非真實影像)。左上:正常站立(綠);右上:正常坐姿(黃);左下:疑似倒地(橘,躺平未滿 3 秒);右下:跌倒倒地(紅,字體放大,右上角統計框轉紅)。淺色底配深色字、深色底配白字,確保看得清楚。

CHAPTER 02

事前準備

2.1 硬體與網路檢查表

- | | |
|---|---|
| ☐ TP-Link Tapo C220(或同系列有雲台的型號)一台以上,已通電並在 Tapo App 顯示在線。 | ☐ 知道每台攝影機的 IP(Tapo App → 攝影機設定 → 裝置資訊)。 |
| ☐ 一台 Windows 電腦(建議 16 GB 記憶體;沒有獨立顯卡也能跑,但多鏡頭會吃 CPU)。 | ☐ 一支手機,用來加 LINE 官方帳號好友、在 Tapo App 設定攝影機帳號。 |
| ☐ 電腦與攝影機連在同一個區域網路、同一個網段(例如都是 192.168.1.x)。 | ☐ 網路不可以把攝影機隔離(見第 8 章:IoT 隔離、訪客網路)。 |

2.2 在 Tapo App 設定「攝影機帳號」

RTSP 與 ONVIF 都要用「攝影機帳號」登入。這組帳密跟你登入 Tapo App 的 TP-Link 雲端帳號不同。

1. 開啟 Tapo App → 點選該台攝影機 → 右上角齒輪(設定)。
2. 進階設定 → 攝影機帳號 → 自訂一組帳號與密碼。
3. 每一台攝影機都要各自設定一次,不會因為別台設過就自動套用。

地雷

- 連線時出現 401 Unauthorized,代表這台攝影機還沒設定攝影機帳號,不是密碼打錯。
- 帳號、密碼只放在一個設定檔(camera_config.py),不要貼到聊天室、不要上傳 GitHub。

2.3 安裝軟體

```
:: 1. 安裝 Python(建議 3.11 或 3.12;老師的電腦用 3.14 也成功,
::  但若 torch 安裝失敗,請改用 3.11 / 3.12)
:: 2. 安裝套件
pip install opencv-python ultralytics onvif-zeep numpy pillow
pip install SpeechRecognition pyttsx3 pycaw
:: 3. 安裝 ffmpeg(語音監聽要用)
winget install Gyan.FFMpeg
:: 4. 確認
python --version
ffmpeg -version
```

注意

- `ultralytics` 會一併安裝 `torch`,檔案很大,安裝到一半不要強制中斷;裝完用 `pip list` 確認真的有這個套件。
- `pycaw` 只用來調整電腦音量,屬於選用。 `pytapo` 不需要安裝(韌體更新後已無法使用)。
- 第一次執行 YOLO 會自動下載 `yolov8n-pose.pt` 模型檔,需要網路。

2.4 用兩行指令確認「連得到」

動手寫程式前,先確認網路層沒問題。這比看程式報錯快很多。

```
# 在 PowerShell 執行,把 192.168.x.x 換成攝影機 IP
Test-NetConnection -ComputerName 192.168.x.x -Port 554 # RTSP
ffprobe -rtsp_transport tcp -show_streams rtsp://帳號:密碼@192.168.x.x:554/stream1
```

- `TcpTestSucceeded : True` → 網路通了,可以進階段 1。
- `False` 或 `ping` 顯示「目的地不可達」→ 先排除網路問題(第 8 章),不要去改程式。
- `ffprobe` 會列出串流內容: `stream1` 有影像 + 音訊, `stream2` 只有影像。這個差異後面語音階段會用到。

2.5 建立專案資料夾與機密檔規則

- 建立一個專案資料夾,之後所有檔案都放這裡;資料夾名稱建議含日期,例如 `115-08-23-居家長照跌倒自動追蹤專案` (民國年-月-日-名稱)。
- 會含有帳密或金鑰的檔案(`camera_config.py` 、 `line_config.py`)只留在自己電腦,分享作業或上傳前先移除或換成範例值。

與 AI 對話的六個原則

這個專案能做出來,靠的不是一次丟一大段需求,而是一句一句把事情講清楚、看結果、再講下一句。以下六個原則貫穿整本手冊。

原則	怎麼做
1. 先說要什麼,不用先想怎麼做	說出想要的結果(「能看到四台鏡頭的畫面」),找哪個檔案、用哪個函式庫是 AI 的工作。
2. 一次只做一個階段	先能動,再求完整。階段驗收通過才進下一階段,出問題時範圍小、好找。
3. 回報「看到的」,不要回報「猜到的」	講哪個按鈕、什麼操作、看到什麼、預期什麼,並貼上錯誤訊息或日誌。診斷交給 AI。
4. 方向、顏色、聲音由你親自確認	「鏡頭往哪邊轉」「字看不看得清楚」「有沒有聽到語音」這類事,AI 只能從截圖間接推測,容易判斷錯;直接告訴它你實際看到(聽到)的結果最快。
5. 先驗證邏輯,再上真機	可以用假資料測的(換算公式、比對邏輯),要求 AI 先寫測試跑過;真實硬體的測試留給最後。
6. 收工前留一手	存一份備份、請 AI 把已校準的數值與踩過的坑寫成紀錄,下次對話才接得起來。

好的回報 vs. 不好的回報

避免 「怎麼怪怪的,你看一下。」 沒說哪個畫面、哪個按鈕、什麼時候發生,AI 只能把整個系統重查一遍。	建議 「按下地下室的停止按鈕後,畫面卡在最後一張沒有變成『串流已停止』的提示。」 一句話就鎖定到特定程式,很快找到原因。
避免 「追蹤不行。」 不知道是完全不動、動太慢、還是動錯方向。	建議 「人站定不動之後,鏡頭會左右來回移動停不下來。」 這句話直接指出「過衝震盪」,AI 就知道要改成比例修正。

提示語裡的符號

本手冊提示語中的【**虛線框**】代表要換成你自己的內容,例如【**攝影機 IP**】。其餘文字可以原樣貼給 AI。

給老師的備註 - 學生用的 AI 助理不同、版本不同,輸出一定會有差異。以「驗收」欄位為準,不以程式碼是否與範例一致為準。 - 手冊內的參數值是老師實測後的結果,學生的攝影機位置、光線、網路不同,需要自己微調。

開場:建立專案與工作方式

目標:讓 AI 知道你的環境與規則,並先拿到一份分步計畫。這一步**不寫程式**。

提示語 0-1 開場

我是【學校 / 科系】的學生,正在做「居家長照跌倒偵測與自動追蹤」專題。環境:Windows 11、Python【版本】,攝影機是 TP-Link Tapo C220,共【數量】台,電腦和攝影機在同一個區域網路。請在資料夾【路徑】建立專案,之後所有檔案都放這裡。

工作方式請照這樣:

1. 一次只做一個功能,做完要**實際執行驗證**(截圖或日誌為證)再回報,不要只說「寫好了」。
2. 帳號、密碼、IP 不要寫死在程式裡,集中放 `camera_config.py`。
3. 遇到必須由我親自確認的事(例如鏡頭實際往哪個方向轉、有沒有聽到聲音),請直接問我,不要用猜的。
4. 每次改完程式,告訴我怎麼重新啟動。

先不要寫程式,告訴我你打算分成哪幾個步驟。

驗收

- AI 回覆了一份分步計畫(大致是:單鏡頭畫面 → 多鏡頭 → AI 偵測 → 雲台 → 警報 → 語音 → 追蹤)。
- 專案資料夾已建立。

想一想

- 如果 AI 的計畫跟本手冊順序不同,可以討論;但「先看到畫面、最後才做追蹤」的順序不要顛倒。
- 提示語 0-1 第 3 點很重要:它讓 AI 養成「該問你就問」的習慣,後面調方向時才不會一直猜錯。

連上單台攝影機看到畫面

目標:用 Python 即時顯示一台攝影機的畫面,斷線能自動重連。

提示語 1-1 單鏡頭檢視

請寫一支 `01_camera_viewer.py`:用 OpenCV 讀取 Tapo 攝影機的 RTSP 串流並顯示,按 q 離開。要求:

- 網址格式 `rtsp://帳號:密碼@IP:554/stream2` (stream2 是低解析度副串流,較省效能)。
- 用 TCP 傳輸:必須在 `import cv2` 之前設定 `os.environ["OPENCV_FFMPEG_CAPTURE_OPTIONS"] = "rtsp_transport;tcp"`。
- 讀取失敗要**自動重連**(等 2 秒再試),不可以讓整個視窗消失。
- 帳號、密碼、IP 放 `camera_config.py` 的 `CAMERAS` 字典(名稱 → ip / username / password),並提供 `rtsp_url`(名稱) 函式。

我的攝影機 IP 是【IP】,攝影機帳號是【Tapo App 設定的攝影機帳號】(密碼我會自己填進 `camera_config.py`,不要貼給你)。寫完請直接執行,並用截圖或日誌確認真的有畫面。

驗收

- 視窗顯示即時畫面,時間戳記每秒在走。
- 拔掉攝影機電源再插回,程式會自動恢復畫面(不用重開)。

地雷與解法

- 401 Unauthorized → 該台攝影機的「攝影機帳號」沒設定或打錯(回 2.2)。
- 連不上、ping 顯示目的地不可達 → 網路被隔離(第 8 章),不是程式問題。
- 花屏、斷幀 → 沒有設定 TCP,或設定放在 `import cv2` 之後(無效)。
- 視窗突然消失 → 讀取失敗時直接跳出迴圈了,要改成重連。

LEVEL 1 · 基礎

2

四分割多鏡頭儀表板

目標:2×2 視窗同時顯示四台鏡頭,每台可獨立啟動、停止、放大。

提示語 2-1 儀表板

請用 Tkinter 寫 `tapo_multi_camera_dashboard.py`,四台鏡頭 2×2 排列。每台一個 `CameraFeed` 類別,各自一個擷取執行緒,UI 用計時器定期取最新畫面顯示(不要在擷取執行緒裡操作 UI)。每張卡片要有:名稱與 IP、FPS、人數徽章(先放 0)、啟動 / 停止按鈕、放大按鈕(雙擊畫面也可以切換放大與四分割),上方工具列要有「全部啟動」「全部停止」。

停止時畫面必須變成「串流已停止」的提示文字,不可以凍結在最後一張影像;停止後再啟動要能恢復。斷線要自動重連。攝影機名稱與 IP 都從 `camera_config.py` 讀取。寫完請實際操作「停止 → 啟動」並截圖給我看。

驗收

- 四台同時出畫面,FPS 約 10–15。
- 按停止 → 該格變成「串流已停止」文字、FPS 歸零;再按啟動 → 畫面恢復。
- 雙擊放大 / Esc 或再雙擊回四分割。

地雷與解法

- 按停止後畫面凍結:擷取執行緒剛好讀完一張新影格、還沒寫入就被 `stop()` 清空,影格又被寫回去。解法:寫入影格前先檢查 `running`,迴圈結束後再清一次,UI 取畫面時也要判斷 `running`。(這是真實發生過的競速問題,提示語已要求「不可以凍結」,AI 通常會自己處理。)
- 同時開四台,最後一台逾時(Stream timeout 30 秒):不是攝影機壞,是前幾台載入模型把 CPU 吃光。單獨重啟那一台,或錯開啟動時間。
- 放大模式要跳過被隱藏鏡頭的繪圖,否則白白浪費 CPU。

AI 姿態偵測:站、坐、跌倒

目標:每台鏡頭標示人數與姿態;躺平持續 3 秒才判定「跌倒倒地」。

提示語 3-1 姿態偵測模組

請新增 `ai_fall_detector.py`,類別 `AIFallDetector`,用 `ultralytics` 的 `yolov8n-pose.pt` 偵測人與 17 個關鍵點,並整合進儀表板。要求:

1. 省 CPU:推論尺寸 `imgsz=256`、信心 0.35;每 4 幀才真正推論一次,其餘幀沿用上次結果畫框,避免標註閃爍。
2. 姿態判斷(COCO 關鍵點:5/6 肩、11/12 髖):算「肩膀中點到髖部中點」連線相對水平的角度;角度小於 25° 且肩髖垂直落差 ≤ 20 像素 $\rightarrow$ 躺平。沒有肩髖關鍵點時,外框長寬比 < 0.55 也算躺平。站立:長寬比 ≥ 1.35 且角度 $\geq 65^\circ$;其餘為坐姿。關鍵點信心門檻 0.25。
3. 持續時間:躺平要持續 3 秒才算「跌倒倒地」,未滿 3 秒顯示「疑似倒地」(避免彎腰撿東西誤判)。
4. 標籤顏色:站立綠、坐姿黃、疑似倒地橘、跌倒倒地紅。注意 OpenCV 是 BGR 順序,紅色要寫成 (0,0,255)。
5. 文字可讀性:淺色背景配深色字、深色背景配白字(用背景亮度自動判斷);「跌倒倒地」的標籤字體要放大。
6. 中文顯示: `cv2.putText` 不能顯示中文,請用 PIL 搭配微軟正黑體繪製。
7. 畫面右上角顯示「空間人數 / 站立 / 坐姿 / 跌倒」統計。卡片標題的人數徽章也要更新。

寫完請先用假的關鍵點資料測試四種狀態的畫面都能正確畫出來(不用等真人演),把四種狀態拼成一張圖給我看。

驗收

- 人站在鏡頭前,顯示綠色「正常站立」;坐下變黃色「正常坐姿」。
- 在安全情況下(墊子上、旁邊有人)躺下:先出現橘色「疑似倒地」計時,滿 3 秒變紅色「跌倒倒地」且字體明顯變大。
- 彎腰撿東西、快速蹲下不會變成紅色。

地雷與解法

- 中文變成問號或方框 $\rightarrow$ 改用 PIL + 中文字型(`C:\Windows\Fonts\msjh.ttc`)。
- 淺黃底配白字看不清 $\rightarrow$ 用亮度判斷文字顏色,不要每種狀態寫死白字。
- 四鏡頭 CPU 滿載、畫面卡 $\rightarrow$ 加大推論間隔(每 N 幀)、縮小 `imgsz`;不要降低信心門檻硬撐。
- 紅色變藍色 $\rightarrow$ 忘了 OpenCV 是 BGR。連「警報框」這種細節也要用截圖驗證,不要只看程式碼。
- 夜間紅外線模式下畫面是灰階,偵測仍可運作,但顏色特徵會失效(見階段 7 的限制)。

雲台控制:ONVIF 與方向校準

目標:每台鏡頭有上下左右、停止、歸位、巡航按鈕;按「▲」鏡頭就是往上看。

提示語 4-1 雲台模組

請新增 `tapo_controller.py`, 用 **ONVIF 協定**(連接埠 2020, 套件 `onvif-zeep`) 控制雲台。**不要用 pytapo**:2026 年 7 月 Tapo 韌體更新後私有協定交握改了, `pytapo` 登入會失敗。

請先寫一小段程式查詢這台攝影機的 PTZ 能力(`GetNodes` 、 `GetPresets` 、 `GetConfigurationOptions` 、 `GetStatus`), 把結果給我看, 確認支援哪些功能(特別是有沒有「歸位」與速度範圍)再開始寫控制功能。

功能:相對步進移動 `relative_move(dx, dy, speed)` (按一下動一小步, 硬體自己煞車)、連續移動 + 停止、歸位、巡航。
在儀表板每張卡片下方加:◀▲▼▶、停止、歸位、巡航。

方向設計成:「按▲鏡頭要往上看」。每台攝影機在 `camera_config.py` 有 `pan_invert` / `tilt_invert` (1 或 -1)可調。
不要自己猜方向:寫完告訴我怎麼測, 我會回報鏡頭實際往哪邊轉, 再由你調整。

方向校準流程(每台鏡頭都要做)

1. 人站在鏡頭視野內, 按一次「▲」。
2. 回報給 AI:「按▲之後, 鏡頭**實際**往下(或往上)轉了。」
3. 若相反, AI 會把該台的 `tilt_invert` 改成 -1(左右同理 `pan_invert`)。
4. 改完**重啟程式**, 再按一次確認。四台各自可能不同(例如掛法不同), 不要假設一樣。

驗收

- 四台各按 ▲▼◀▶, 鏡頭轉動方向與按鈕箭頭一致。
- 停止鍵可以立即煞車; 巡航結束後鏡頭能回到大致正面。

地雷與解法

- 改了 `tilt_invert` 完全沒反應 → 專案裡有不只一份 `camera_config.py` (例如根目錄一份、`scripts` 資料夾一份), 程式實際載入的是另一份。請 AI 在程式裡印出 `camera_config.__file__` 確認載入哪份, 並只保留一份。(這個坑老師耗掉整整一節課。)
- 不要用截圖肉眼判斷方向: 畫面特寫太近、IR 灰階時很難分辨「往上還是往下」, 直接回報你看到鏡頭怎麼轉最準。
- ONVIF 的 y 軸正負號在不同指令(連續移動 / 相對移動)的物理意義不保證一致, 所以不要拿一個指令的結果去推另一個, 一律以「實際轉動」為準。
- 程式碼逾時被吞掉: 雲台指令放在背景執行緒, 失敗時至少要 `print` 出錯誤, 否則你永遠不知道為什麼沒動。

進階挑戰: 修正「歸位」

- Tapo 查詢結果通常是 `HomeSupported = False` 且沒有任何預設點, 若「歸位」用 `AbsoluteMove(0, 0)`, 鏡頭可能轉去貼著牆壁看(座標 0,0 不是「面向房間」)。
- 正確做法: 先手動把鏡頭調到滿意角度, 用 `SetPreset` 存成預設點, 歸位時改呼叫 `GotoPreset`。請 AI 幫你實作並用截圖驗證。

跌倒警報流程與 LINE 通知

目標:判定跌倒時自動拍照、LINE 通知、播放安撫語音;按「解除警報」才恢復偵測。

5.1 先準備 LINE 通道(由你自己操作)

1. 到 **LINE Official Account Manager**(manager.line.biz)建立官方帳號 → 完成手機簡訊認證(免費)。
2. 在該帳號啟用 **Messaging API**,到 LINE Developers Console 取得 **Channel access token(long-lived)**。
3. 用手機掃該帳號 QR Code 加為好友(broadcast 只會發給已加好友的人)。
4. 把 token 放進 `line_config.py` (只留在自己電腦)。

界線:這幾件事 AI 不會、也不應該替你做

- 建立帳號、輸入密碼 / 簡訊驗證碼、綁定金流。就算你明確要求,AI 助理也會拒絕並教你怎麼做——這是保護你的帳號安全,不是 AI 偷懶。
- 在 LINE Developers 建立頻道時,類型要選 **Messaging API**(從官方帳號啟用);直接在 Developers Console 建的「LINE Login」頻道**不能**用來推播。

5.2 請 AI 做警報流程

提示語 5-1 警報流程

請在儀表板加入「跌倒警報流程」。當某台鏡頭第一次被判定「跌倒倒地」時,依序做:

1. 在 `emergency_snapshots/` 存一張快照,檔名含鏡頭名稱與時間,影像上標示警報時間。
2. 用 LINE Messaging API 的 broadcast 發一則通知(含鏡頭名稱、時間、空間人數)。**推播請用獨立子行程** (`line_worker.py`),並用標準函式庫 `http.client`,不要用 `requests` ——有些電腦上 `requests` 呼叫外部 HTTPS 會卡死不理 `timeout`。
3. 鏡頭自動往下微轉,看向地面。
4. 用 `pyttsx3` 在電腦播放安撫語音(先做成可開關,預設關閉)。

用 `is_emergency` 旗標避免同一次跌倒重複通報。**一定要同時提供「解除警報」按鈕:**平常灰色不可按,警報中變紅色可按,按下才把旗標重設、恢復偵測下一次跌倒。LINE 失敗(例如額度用完回 HTTP 429)要把錯誤印出來,不可以整個程式當掉。測試時請不要真的發 LINE,先用假的函式代替,我確認流程正確後再接真的。

驗收

- (用假推播)模擬跌倒 → 資料夾多一張快照、日誌顯示「準備推播」、卡片「解除警報」變紅。
- 按「解除警報」→ 按鈕變灰、旗標重設;再模擬一次跌倒能再次觸發。
- (接真的 LINE)手機收到一則通知。測試次數要節制。

地雷與解法

- 只通報一次,之後再怎麼跌都沒反應 → `is_emergency` 設成 True 後沒有重設機制。這就是要「解除警報」按鈕的原因。
- HTTP 429 "You have reached your monthly limit" → 免費方案本月額度用完(測試時狂觸發很容易用光)。處理:等下個月、升級方案,或另開一個專屬的 LINE 官方帳號並換 token。
- 換新 LINE 帳號後收不到 → 新帳號的好友要重新加,舊帳號的好友不會自動搬過去。
- 程式在主執行緒呼叫雲台 `time.sleep` 會卡住擷取 → 雲台動作放背景執行緒。

語音求救:用攝影機自己的麥克風

目標:對著任一台鏡頭喊「救命」,系統知道是哪個房間,並觸發同一套警報。

先搞懂兩個事實(用指令驗證,不要相信文字)

- **stream2 完全沒有音訊軌**,只有 **stream1 有**(`pcm_alaw`、8 kHz、單聲道)。用 `ffprobe` 分別檢查兩條串流即可親眼確認。語音監聽必須接 `stream1`,否則 `ffmpeg` 會一直等不到音訊而卡住。
- 若用 `speech_recognition.Microphone()`,聽的是**電腦本機的麥克風**,不是攝影機的;對著攝影機喊,電腦根本聽不到。

提示語 6-1 語音監聽

請替每台攝影機各加一個「語音監聽」背景執行緒,並在工具列加一顆總開關「語音求救:開啟 / 關閉」(預設關閉)。要求:

1. 用 `ffmpeg` 讀取該台的 RTSP `stream1`(`-vn -acodec pcm_s16le -ac 1 -ar 16000 -f s16le pipe:1`),每 3 秒一段。
2. 巨響偵測:該段音量 RMS 大於 0.09 → 視為巨響 / 尖叫,觸發警報。
3. 關鍵字偵測:用 `SpeechRecognition` 的 `recognize_google(language="zh-TW")` 轉文字,命中關鍵字清單(必須含「求救」「救命」「救我」「幫我」「跌倒」「好痛」「起不來」「緊急」等)就觸發警報。每次辨識到的文字都要 `print` 出來,方便除錯。
4. 觸發後走跟跌倒警報**同一套流程**(拍照、LINE、安撫語音),訊息裡要寫明是哪一台鏡頭。觸發後休息數秒避免連發。

寫完請先用 `ffprobe` 證明 `stream1` 有音訊軌,並抓 3 秒音訊算出 RMS 給我看,再讓我實際對鏡頭講話測試。

驗收

- 日誌裡出現「[某鏡頭聽見]:「...」」,代表語音辨識通了。
- 對著鏡頭說「救命」→ 卡片「解除警報」變紅,並有拍照 / 語音 / (LINE)。
- 安靜時不會一直誤觸發。

地雷與解法

- 日誌有聽到「求救」卻沒觸發 → 關鍵字清單漏了這個詞本身(辨識正確 ≠ 命中清單)。這是真實發生過的 bug。
- 完全沒聲音、`ffmpeg` 卡住 → 接到 `stream2` 了。
- 總開關預設關閉,重啟程式後要再按一次(也可以請 AI 改成記住設定)。
- 語音辨識需要網路(Google 服務),斷網時只剩巨響偵測。
- 無法從攝影機喇叭播放聲音:實測 ONVIF 的 `GetAudioOutputs()` 回空值、`deviceio` 服務被攝影機拒絕——對講走的是 TP-Link 私有協定,目前不通。安撫語音只能從電腦喇叭播放。(這是「查證後承認做不到」的好範例:不要硬做,寫進限制。)

跌倒鎖定追蹤

目標:跌倒後鏡頭自動把人拉回畫面中央;人站起來走動也持續跟著同一個人;可用按鈕或舉右手停止。

提示語 7-1 鎖定追蹤(一次講完設計)

請替 CameraFeed 加入「跌倒鎖定追蹤」。某人被判定「跌倒倒地」後,鏡頭要自動鎖定追蹤他,直到我按「解除追蹤」或他舉起右手。設計要求:

1. **認得同一個人:**鎖定當下擷取他人物外框的 **HSV 顏色直方圖**(H、S 兩個通道)當特徵。之後不管他站起來、坐下、走動,都用 `cv2.compareHist` (Bhattacharyya 距離,小於 **0.6** 才算同一人)在畫面中找回他。不可以因為姿態不再是「跌倒」就跟丟,也不可以追到別人。每次成功比對後用最新畫面更新特徵。
2. **移動方式:比例修正**,不要固定步伐。算人物中心相對畫面中心的偏移(-1 ~ +1);偏移的絕對值在 **0.18** 以內(死區)不動;超出才動,移動量 = 偏移 × 最大步長 **0.12**。雲台馬達速度用硬體上限 **1.0**。每 **0.2 秒** 最多修正一次。
3. **方向:**直接沿用方向鍵按鈕(階段 4)已校準好的 `ptz_step` 慣例,不要重新猜正負號。人在畫面右邊 → 往右轉;在下方 → 往下轉。
4. **兩種解除追蹤方式**(跟「解除警報」是分開的兩件事,警報仍保留):
 - 卡片上的「解除追蹤」按鈕(正在追蹤時才亮起)。
 - 被追蹤者**舉起右手**:COCO 關鍵點 10(右手腕)的 y 座標比關鍵點 6(右肩)的 y 小 20 像素以上,且兩點信心都 > 0.25。但人還躺著(跌倒倒地狀態)時不要檢查舉手,因為躺姿手腕常常看起來比肩膀高,會誤判。
5. 按「解除警報」要同時清除追蹤特徵,並讓下一次新警報能重新追蹤。
6. 追蹤的每一步都要 `print` 日誌(鎖定、偏移量、決定的移動、不需要動、找不到人),並且**不可以把例外靜默吞掉**。

請先用假資料寫單元測試驗證:(a)人在左 / 右 / 上 / 下 / 中央時算出的移動方向;(b)畫面有兩個穿不同顏色的人,其中一個站起來走到另一側時仍追對人;(c)躺著手腕高於肩膀不會誤判舉手、站著舉手會解除追蹤;(d)偏移越接近中心修正量越小。測試全過再讓我用真機試。

驗收(真機)

- 躺下滿 3 秒觸發跌倒 → 日誌出現「鎖定跌倒者,記錄衣服顏色特徵」,鏡頭開始把你拉向畫面中央。
- 站起來、走到視野邊緣 → 鏡頭持續跟著你(不會因為不再是「跌倒」就停)。
- 站定不動 → 鏡頭收斂停下,不會左右來回晃。
- 站著舉右手,或按「解除追蹤」→ 鏡頭停止跟拍;「解除警報」按鈕仍然是紅的。

7.1 調整的對話(迭代才是常態)

老師版本在同一個晚上連續調了好幾輪,實際對話如下,請學習「怎麼回報」:

你說	發生了什麼 / AI 怎麼處理
「有稍微動一下」	追蹤有動兩步就停了。日誌顯示「警報中但沒有跌倒中的人」——人一站起來就被判定不再是跌倒,舊邏輯只追跌倒者。→ 改成顏色特徵鎖定。
「追蹤速度要再加快」	檢查間隔 1 秒 → 0.5 秒、步長加大、馬達速度 0.5 → 0.8 → 1.0。同時新增「解除追蹤」按鈕與舉右手手勢。
「這次不會追蹤」	日誌顯示剛鎖定就「偵測到舉起右手」。躺姿被誤判成舉手。→ 躺著時不檢查舉手。
「雲台速度可以再快嗎」	馬達速度已是硬體上限 1.0(ONVIF 範圍 0~1),只能加大步長(0.12 → 0.2)與縮短檢查間隔(0.25 → 0.2 秒)。

你說	發生了什麼 / AI 怎麼處理
「人定點不動後,雲台會左右來回移動」	固定步長 0.2 太大,衝過頭又反向修正。→ 改成比例修正(最大步長 0.12)+ 加大死區(0.15 → 0.18)。

真正的反應速度天花板

- AI 推論每 4 幀才跑一次(15 FPS 時約 0.27 秒才更新一次人的位置)。檢查頻率再高,位置資料也不會更新更快;想更即時要縮短推論間隔,但 CPU 負擔會增加。
- 「加快」的手段有限且互相牽制:馬達速度(已到頂)、步長(太大會震盪)、檢查間隔(受推論頻率限制)。調一個參數前先問 AI「現在每個參數是多少、各自的作用」。

已知限制(要誠實寫進你的報告)

- 夜間紅外線模式是灰階,衣服顏色特徵幾乎失效,追蹤準確度會下降。
- 多人穿相近顏色時可能追錯;人被家具完全遮住時會暫停追蹤。
- 一台鏡頭視野有限,人走出視野就追不到了(需要跨鏡頭接力,屬於延伸題)。

LEVEL 3 · 挑戰

8

備份、紀錄與驗收

目標:專案可以被另一個人(或下週的你)接手繼續做。

提示語 8-1 另存備份

功能都穩定了。請先把整個專案資料夾完整複製一份,新資料夾名稱為【民國年-月-日-專案名稱】,原資料夾保留不動,複製完列出新資料夾的檔案清單與大小讓我確認。

提示語 8-2 留下紀錄

請把今天的做法與踩過的坑整理成一份紀錄(不要貼程式碼,也不要寫任何帳號、密碼或 token),方便下次接著做。重點只寫「看程式碼看不出來」的事:

- 已經校準好的數值(例如每台鏡頭的 `pan_invert` / `tilt_invert`、各項門檻),以及當初怎麼確認的。
- 踩過的坑與根本原因(例如多份設定檔、stream2 沒音訊、LINE 額度)。
- 尚未完成的事與下一步(例如換新的 LINE 通道、修正歸位)。

提示語 8-3 安全檢查

請檢查整個專案資料夾,列出所有含有帳號、密碼、token 或 IP 的檔案,說明哪些絕對不能上傳或分享,並幫我建立一份 `.gitignore` 與一份「可分享版」的範例設定檔(把機密換成假值)。

8.1 期末驗收檢查表

- | | |
|--|--|
| ☐ 四台鏡頭穩定直播 10 分鐘不當機。 | ☐ LINE 通知收得到(或已說明額度限制)。 |
| ☐ 停止 / 啟動、放大、全部啟動 / 停止都正常;四種姿態標籤清楚。 | ☐ 語音「求救 / 救命」能觸發,日誌可看到辨識文字。 |
| ☐ 雲台四個方向與按鈕一致。 | ☐ 鎖定追蹤:躺下 → 站起 → 走動持續跟隨。 |
| ☐ 跌倒 3 秒才判定,彎腰不誤報。 | ☐ 站定後鏡頭不會來回晃;舉右手與按鈕都能解除追蹤。 |
| ☐ 警報會拍照,「解除警報」可重複使用。 | ☐ 已另存備份並寫好紀錄;專案資料夾不含真實帳密。 |
| | ☐ 報告中寫出至少三項「已知限制」。 |

提示語句型庫

下面這些句型跟「居家長照」無關,可以套用在任何專題。

要求先驗證

做完請先實際執行,並用**截圖或日誌**證明有效,再告訴我結果;不要只說「寫好了」。

回報問題(填空)

【哪個畫面 / 按鈕】在【什麼操作】的時候,出現【我看到什麼】,我預期是【預期結果】。日誌 / 錯誤訊息如下:
【貼上】

需要我親自確認的事

這件事需要我親自確認(方向 / 顏色 / 聲音)。請告訴我怎麼測試,我會回報**實際看到(聽到)**的結果,你再依據調整,不要用猜的。

先測邏輯再上真機

先用假資料寫測試驗證這段邏輯,列出每個測試案例與結果,全部通過再部署到真實硬體。

調整參數

目前【功能】的症狀是【太慢 / 太快 / 會來回晃】。請先列出相關的所有參數、目前值與各自的作用,說明哪些已經到上限,再建議怎麼調。

改了沒變化

我改完沒有任何變化。請先確認三件事:程式有沒有重新啟動、程式實際載入的是哪一個檔案(印出 模組.__file__)、專案裡有沒有同名的重複檔案。

看不懂時

請用高中生聽得懂的方式,逐段說明這支程式在做什麼;說完請問我三個問題,確認我真的理解。

懷疑做不到時

在動手寫之前,請先用小實驗確認這個功能在我的設備上**技術上做得到**,把實驗結果給我看。如果做不到,直接告訴我原因,不要硬做一個假的。

收尾

(1) 先把專案另存一份,資料夾名稱【日期-名稱】。(2) 把今天的做法、已校準的數值、踩過的坑與下一步整理成紀錄。

除錯速查表

遇到問題先對照這張表,先判斷是網路、設定、還是程式,再決定要不要動程式碼。

症狀	最可能的原因	處理
RTSP 回 401 Unauthorized	該台攝影機沒設定「攝影機帳號」,或打成雲端帳號	Tapo App → 進階設定 → 攝影機帳號,每台各設一次。
ping「目的地不可達」、554 埠連不上	路由器的 IoT 網路隔離 / 訪客網路,把攝影機獨立放到隔離網段	到路由器 App(例如 Deco)關閉 IoT 隔離;確認電腦與攝影機同一個 SSID。不要改程式。
ping 純逾時(連「不可達」都沒有)、IP 是別的網段	攝影機實際在另一個網路(IP 網段不同),電腦沒有路由到那邊	把攝影機連回與電腦相同的 Wi-Fi,拿到同網段 IP;更新設定檔。
畫面花屏、斷幀	RTSP 走 UDP	設定 <code>rtsp_transport;tcp</code> ,且要在 <code>import cv2</code> 之前。
同時開多台,最後一台 30 秒逾時	前幾台載入模型吃光 CPU,握手被排擠	單獨重啟那一台,或錯開啟動。
中文標籤變問號	<code>cv2.putText</code> 不支援中文	改用 PIL + 微軟正黑體。
按停止後畫面凍結	擷取執行緒的競速寫入	寫入前檢查 <code>running</code> ,結束後再清一次。
按▲鏡頭往下	<code>tilt_invert</code> 方向不對,或改到沒被載入的設定檔	回報實際方向 → 改 -1 → 重啟;印出 <code>__file__</code> 確認載入哪份。
按「歸位」後鏡頭看牆壁	<code>AbsoluteMove(0,0)</code> 不是面向房間的位置	改用 <code>SetPreset</code> / <code>GotoPreset</code> 。
LINE 失敗,429	免費額度用完	等下月、升級,或換專屬官方帳號與 token。
LINE 呼叫卡住不回應	<code>requests</code> / <code>urllib3</code> 在某些電腦被資安軟體卡住	改用 <code>http.client</code> ,並用獨立子行程送出。
只通報一次就不再通報	<code>is_emergency</code> 沒有重設機制	加「解除警報」按鈕重設旗標。
喊「求救」沒反應(日誌也沒有)	語音總開關沒開;或接到 stream2(沒音訊)	打開開關;音訊改接 stream1(用 <code>ffprobe</code> 驗證)。
日誌有辨識到「求救」但沒觸發	關鍵字清單沒有這個詞	補齊清單。
追蹤站定後左右來回晃	固定步長衝過頭	改比例修正、加大死區。
追蹤剛鎖定就停了	躺姿被誤判成舉手	人還躺著時不檢查舉手手勢。
人站起來後追蹤停止	只追「跌倒狀態」的人	改用顏色特徵追蹤同一人。
改了程式沒變化	沒重啟(沒有熱重載),或改到別份檔案	完整關閉後重新啟動;確認實際載入的檔案。

關鍵參數表與已知限制

這些是老師版本實測後的數值,可當起點。每個人的鏡頭位置、光線、網路不同,請自己再微調,並請 AI 說明每個參數的作用。

參數	數值	意義 / 調整方向
跌倒確認時間	3.0 秒	躺平持續多久才算跌倒。縮短更靈敏但誤報增加。
偵測信心	0.35	低於此值的人不算。
推論尺寸 imsz	256	越小越快、越不準。
推論間隔	每 4 幀	CPU 吃緊時加大;想更即時就縮小。
關鍵點信心	0.25	判斷肩、髖、手腕是否可用。
躺平判定	$<25^{\circ}$	肩—髖連線角度,且肩髖垂直落差 ≤ 20 像素;或長寬比 < 0.55 (無肩髖時)。
站立判定	$\geq 65^{\circ}$	且外框長寬比 ≥ 1.35 ;或肩髖落差 > 60 像素且角度 $\geq 70^{\circ}$ 。
文字對比門檻	亮度 140	背景亮度高於此用深色字。
追蹤檢查間隔	0.2 秒	受推論頻率限制,再小意義不大。
追蹤死區	0.18	偏移(相對半個畫面)小於此不動。越大越穩、越不靈敏。
最大步長	0.12	比例修正的上限。太大會震盪。
雲台速度	1.0	硬體上限(ONVIF 速度範圍 0~1)。
顏色比對門檻	<0.6	Bhattacharyya 距離,越小越像。
舉手判定	腕比肩高 20px	右手腕 $y < \text{右肩 } y - 20$;躺著時不判。
音量門檻 RMS	0.09	巨響 / 尖叫。環境吵就調高。
語音分段	3 秒	16 kHz、單聲道。
標籤字級	15 / 26	一般 / 跌倒倒地。

已知限制(誠實寫進報告)

- 本系統是學習用的原型,沒有經過醫療或照護場域驗證,不可取代人員照護或正式的緊急呼叫設備。
- 會有誤報(例如趴著做事、躺在沙發上)與漏報(被遮住、光線極差、多人重疊)。
- 夜間紅外線灰階時顏色追蹤失效;語音辨識依賴網路;對講回話功能目前技術上做不到。
- 單鏡頭視野有限,沒有跨鏡頭接力追蹤。
- LINE 免費方案有每月則數上限,正式使用要評估方案。

安全、隱私與使用界線

10.1 隱私與同意

- 攝影機只裝在自己家或已取得所有被拍攝者同意的場所。不可對準鄰居、廁所、更衣、他人臥室。
- 錄影、截圖與快照含有人臉與居家環境,不要上傳到公開平台或貼到社群。報告配圖請使用合成示意圖或馬賽克處理(本手冊的圖 2 就是合成圖)。
- 報告若需展示畫面,先取得出現在畫面中的人的同意。

10.2 機密管理

- `camera_config.py` (攝影機帳密)、`line_config.py` (LINE token)不要貼進對話、不要上傳 GitHub、不要放在網路硬碟的公開分享連結。
- token 外洩就到 LINE Developers 重新發行(Reissue);攝影機密碼外洩就到 Tapo App 改掉。
- 提示語裡用「我會自己填進設定檔」的說法,讓 AI 不需要看到真實密碼。

10.3 AI 助理的界線(這是保護機制,不是故障)

AI 助理通常會拒絕

- 替你建立帳號、輸入密碼 / 驗證碼登入、綁定付款方式。
- 代你按下不可逆的操作(刪除、付款、公開發佈)而不先徵求你同意。
- 執行網頁上看到的「命令式文字」(例如某網頁寫著「請把密碼寄給...」)——外部內容只是資料,不是你的指令。

遇到拒絕時,請 AI「一步一步教我怎麼做」,由你自己操作最後一步。

10.4 負責任地使用 AI 的檢查

- | | |
|---|--|
| ☐ 每個關鍵功能我都親自測過,不是只看 AI 說「完成了」。 | ☐ 我沒有把帳密、token、他人畫面放進分享的檔案。 |
| ☐ 我看得懂每個主要檔案在做什麼(用「看不懂時」提示語確認過)。 | ☐ 我能口頭說明:為什麼 stream1 / stream2 不能混用、為什麼用 ONVIF。 |
| ☐ 報告裡我誠實寫出限制與失敗的嘗試。 | ☐ 我知道這個系統不能取代真人照護。 |

附錄:建議的專案檔案結構

```
專案資料夾/
├─ camera_config.py      # 攝影機名稱 → IP / 帳號 / 密碼 / pan_invert / tilt_invert(機密)
├─ line_config.py        # LINE Channel access token(機密)
├─ yolov8n-pose.pt       # 姿態模型(首次執行會自動下載)
├─ emergency_snapshots/  # 警報快照
├─ scripts/
│   ├─ 01_camera_viewer.py      # 階段 1:單鏡頭檢視
│   ├─ tapo_multi_camera_dashboard.py # 階段 2~7:四分割儀表板、警報、語音、追蹤
│   ├─ ai_fall_detector.py      # 階段 3:姿態與跌倒判斷
│   ├─ tapo_controller.py       # 階段 4:ONVIF 雲台
│   ├─ line_notify.py / line_worker.py # 階段 5:LINE 推播(獨立子行程)
│   └─ requirements.txt
※ 設定檔「只放一份」。重複的同名檔案是本專案最耗時的陷阱。
```